

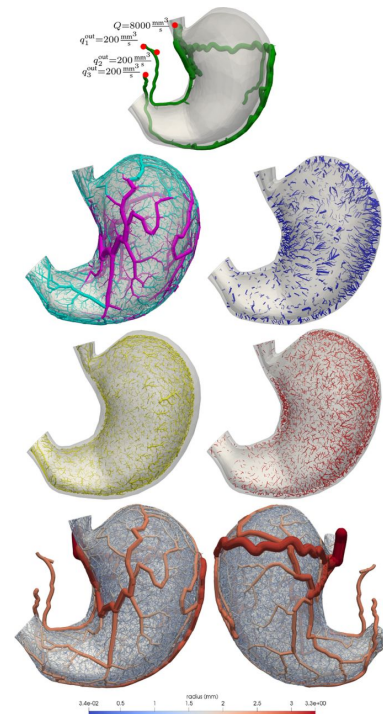
Projeto Temático: TriCCO

Árvores com Trifurcações

Maria Eduarda Bessa Teixeira

Arthur Alexandre Cunha Custodio

Algoritmos e Programação Avançada



CCO: Conceito e Princípios

Modelo Construtivo

A árvore é formada incrementalmente, adicionando um terminal (folha) por vez. Cada novo terminal é conectado à estrutura existente, expandindo a rede de forma progressiva.

Otimização

A cada passo, o novo segmento é conectado de forma a **minimizar o custo total** da árvore. O algoritmo busca o ponto de conexão que resulta no menor aumento de custo global.

Custo

Representa a energia necessária para manter o volume da rede e transportar o fluido. É composto por dois componentes: custo de manutenção (proporcional ao volume) e custo de transporte (proporcional à resistência hidrodinâmica).

Restrição (Lei de Murray)

O raio de cada vaso é determinado pelo fluxo que ele transporta, garantindo a eficiência hidrodinâmica. Esta lei estabelece que a soma dos cubos dos raios dos filhos é igual ao cubo do raio do pai.

Objetivo Final

Gerar uma rede que perfure o domínio de forma eficiente, com o menor custo possível, mantendo a conectividade entre a raiz e todos os terminais.

Objetivos do Projeto

Objetivo Geral (TriCCO)

- ▶ Expandir o modelo CCO clássico (binário) para permitir trifurcações (até 3 filhos por nó)
- ▶ Explorar novas topologias vasculares potencialmente mais eficientes que o modelo binário.

Objetivos do TP1 (Etapa Atual)

- ▶ Implementar o Núcleo CCO Binário funcional e otimizado
- ▶ Garantir a correta aplicação da **Lei de Murray** e a formulação de **Custo**
- ▶ Exportar a árvore para o formato .vtk para visualização e análise no Paraview.
- ▶ Validar a estrutura e os cálculos com testes de 10 a 250 terminais.

O Coração do CCO: A Função Custo

Custo Total

A soma do custo de manutenção (volume) e do custo de transporte (perda de pressão). O algoritmo CCO busca minimizar ao adicionar terminais.

Custo de Manutenção (Volume)

Proporcional ao volume total da rede. Quanto maior o volume, maior o custo.

$$C_{\text{manut}} \propto \sum L_i \cdot r_i^2$$

Custo de Transporte (Perda de Pressão)

Proporcional à resistência hidrodinâmica. Vasos mais finos e longos têm maior resistência.

$$C_{\text{transp}} \propto \sum (L_i \cdot F_i) / r_i^4$$

Custo Simplificado (Lei de Murray)

Com a Lei de Murray, o custo total fica proporcional ao volume total, simplificando a otimização.

$$C_{\text{total}} \propto \sum L_i \cdot r_i^Y$$

Minimização de Custo: O Processo Otimizado

1. Novo Terminal (q)

Um ponto **q** é amostrado aleatoriamente no domínio. Este ponto representa o novo terminal que será adicionado à árvore.

2. Iteração sobre Segmentos Candidatos (S_cand)

O algoritmo itera sobre **todos os segmentos existentes** da árvore. Para cada segmento, calcula-se o custo de conectar **q** através daquele segmento.

3. Ponto de Conexão Ideal (p_conn)

Para cada segmento candidato, o ponto de conexão ideal **p_conn** é encontrado. Este é o ponto no segmento que **minimiza a distância até q**, reduzindo assim o comprimento do novo vaso.

4. Otimização Local: distPointToSegment

A função **distPointToSegment(q, a, b)** projeta **q** no segmento **ab** e retorna a menor distância. Esta projeção é **crucial para minimizar o custo**, pois reduz o comprimento do novo segmento.

5. Otimização Global: Seleção do Melhor Par

O algoritmo escolhe o par (**S_cand**, **p_conn**) que resulta no **menor aumento de custo total** (**ΔC_total**) para toda a árvore. O custo total é calculado como:

$$\Delta C_{\text{total}} = C_{\text{novo_segmento}} + C_{\text{recalculo_montante}}$$

Onde "**C_novo_segmento**" é o custo do novo vaso (proporcional a seu volume) e "**C_recalculo_montante**" é o custo adicional de recalcular os raios dos nós a montante, mantendo a Lei de Murray.

Arquitetura do Código

A Base Matemática (geometry.cpp)

norm(v)

Calcula o comprimento (magnitude) de um vetor. Essencial para determinar distâncias e magnitudes em operações geométricas.

dist(a, b)

Calcula a distância euclidiana entre dois pontos. Usada para medir proximidade entre nós e segmentos, fundamental para a otimização.

lerp(a, b, t)

Interpolação linear entre dois pontos. Encontra pontos ao longo de um segmento (parâmetro $t \in [0,1]$). Essencial para localizar o ponto de conexão ideal.

distPointToSegment(p, a, b)

Função Crítica. Calcula a menor distância de um ponto p a um segmento ab , retornando também a projeção de p no segmento. Esta função é o núcleo da otimização do CCO, pois determina o ponto de conexão que minimiza o comprimento do novo vaso, reduzindo assim o custo total.

A Estrutura de Dados: Modelando a Árvore

Classe Tree

Objeto central que encapsula a lógica do CCO, o domínio e a lista de nós.

Responsabilidades

- ▶ Gerenciar a lista de nós (`std::vector<Node>`)
- ▶ Controlar o domínio de perfusão (círculo de raio R)
- ▶ Executar o algoritmo de adição de terminais
- ▶ Recalcular fluxo e raio após cada mudança

Estrutura Node

Unidade fundamental que representa um ponto na árvore vascular.

Topologia

- ▶ **id**: Identificador único do nó
- ▶ **parent**: Índice do nó pai (-1 para raiz)
- ▶ **children**: Vetor de índices dos filhos

Geometria e Física

- ▶ **p**: Coordenadas (x, y) no espaço
- ▶ **flow**: Fluxo que passa pelo segmento
- ▶ **radius**: Raio do segmento (Lei de Murray)

Nós Iniciais

- **Nó 0 (rootTop)**: Raiz geométrica na borda do domínio
- **Nó 1 (center)**: Primeira bifurcação física, no centro

Física do Modelo: Lei de Murray

Cálculo de Fluxo (dfsFlow)

- ▶ O fluxo é calculado de baixo para cima (*bottom-up*).
- ▶ Folhas têm fluxo = 1.0.
- ▶ Nós pais: Fluxo = Soma dos fluxos dos filhos.

Lei de Murray ($\gamma = 3.0$)

A lei estabelece que a soma dos cubos dos raios dos filhos deve ser igual ao cubo do raio do pai:

$$\sum r_{\text{filho}}^3 = r_{\text{pai}}^3$$

Em termos matemáticos, isso significa que o raio do vaso pai cresce de forma não linear em relação aos raios filhos

Por que $\gamma = 3.0$?

- ▶ Este valor é ótimo para minimizar o custo total (manutenção + transporte).
- ▶ Resulta em uma rede que equilibra volume e resistência hidrodinâmica.

Fórmula Implementada

$$r_i = r_{\text{raiz}} \times (\text{fluxo}_i / \text{fluxo}_{\text{raiz}})^{1/\gamma}$$

Esta fórmula é aplicada a cada nó para calcular seu raio com base no fluxo que passa por ele.

O Algoritmo CCO em Ação

1

Amostragem

Gera um novo ponto terminal q aleatoriamente no domínio circular.

2

Validação de Distância

Verifica se q está a uma distância mínima dos vasos existentes, evitando sobreposição.

3

Busca Otimizada: distância geométrica

Encontra o segmento que está na menor distância do ponto q .

4

Criação da Bifurcação

O segmento escolhido é dividido ao meio, criando um novo nó de bifurcação. O filho original é reconectado a este novo nó.

5

Conexão do Terminal

O novo ponto q é conectado como um novo nó terminal, filho do nó de bifurcação.

6

Recálculo de Fluxo e Raio

O fluxo e o raio de todos os nós a montante são atualizados para refletir a nova topologia e manter a Lei de Murray.

Validação Matemática

Parâmetros da Simulação (5 Folhas)

$\gamma = 3.0$

$r_{\text{raiz}} = 0.05$

$\text{fluxo}_{\text{raiz}} = 5$

Validação do Fluxo

- ✓ Nó 4: Filhos (1 e 5) têm fluxo=1

$1 + 1 = 2$

- ✓ Saída do programa: **flow=2**

Validação do Raio (Lei de Murray)

- ✓ Nó de Folha (fluxo=1):

$r = 0.05 \times (1/5)^{1/3} \approx 0.02924$

- ✓ Saída: **0.0292402**

- ✓ Nó Intermediário (fluxo=2):

$r = 0.05 \times (2/5)^{1/3} \approx 0.03684$

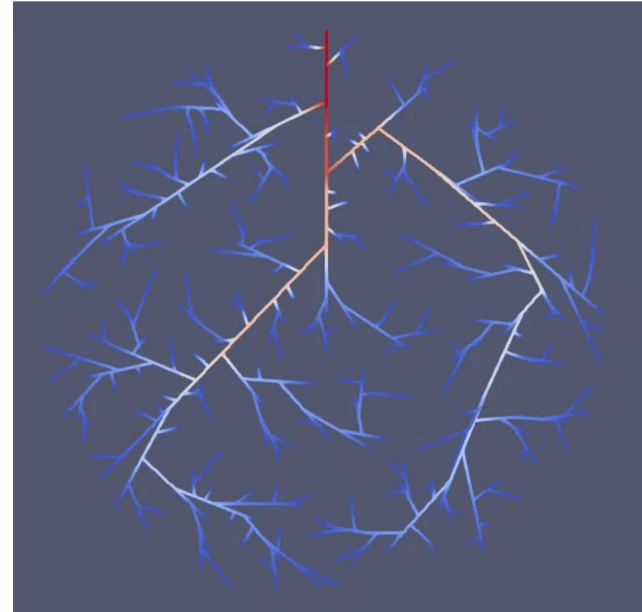
- ✓ Saída: **0.0368403**

✓ Os cálculos de fluxo e raio estão sendo feitos conforme a teoria do CCO descreve. Implementação correta.

```
[TP1] leaves=5 nodes=10 cost=5.35167e-06 depth≈4
node 0 parent=-1 children=1 flow=5 radius=0.05
node 1 parent=4 children=0 flow=1 radius=0.0292402
node 2 parent=8 children=2 flow=4 radius=0.0464159
node 3 parent=6 children=0 flow=1 radius=0.0292402
node 4 parent=2 children=2 flow=2 radius=0.0368403
node 5 parent=4 children=0 flow=1 radius=0.0292402
node 6 parent=2 children=2 flow=2 radius=0.0368403
node 7 parent=6 children=0 flow=1 radius=0.0292402
node 8 parent=0 children=2 flow=5 radius=0.05
node 9 parent=8 children=0 flow=1 radius=0.0292402
```

Resultados do TP1

- ✓ Núcleo CCO binário foi implementado com sucesso, funcional e otimizado.
- ✓ O código foi validado matematica e estruturalmente, com prova de conceito usando exemplo rastreável de 5 folhas.
- ✓ A exportação para .vtk permite a análise visual detalhada da morfologia da árvore no ParaView.
- ✓ Testes com 5 e 250 terminais mostram que o algoritmo é robusto, escalável e gera estruturas vasculares eficientes.



Árvore com 250 terminais: Prova de escalabilidade e eficiência

Próximos Passos: TP2

1

Trifurcações

Expandir o modelo para permitir até 3 filhos por nó, explorando novas topologias vasculares potencialmente mais eficientes.

2

Análise Comparativa

Comparar árvores binárias vs. trifurcadas em métricas importantes.

- Custo total final
- Profundidade da árvore
- Eficiência vascular

3

Relatório Final

Documentar a exploração de novas possibilidades de topologia vascular e conclusões do projeto TriCCO.

Referências:

- <https://github.com/vascular-networks/OpenCCO>
- <https://journals.physiology.org/doi/10.1152/jgp.1926.9.6.835>
- https://www.researchgate.net/publication/224686370_Computer-optimization_of_vascular_trees
- <https://vtk.org/wp-content/uploads/2015/04/file-formats.pdf>